План разработки Space Universe

Вдохновлено: Dual Universe
Движок: Godot 4.7.2
Дата: 11 ноября 2026

> **Важно:** полный аналог Dual Universe (единый шард, MMORPG, массовое строительство, экономика в реальном времени) — проект уровня большой студии. Ниже — реалистичный план: сначала играбельный MVP, затем поэтапное расширение.

1. Видение и рамки

- **Название:** Space Universe
- **Жанр:** космический симулятор с исследованием, строительством и экономикой (на старте — single-player/кооп малого масштаба).
- **Столпы:** полёт в космосе, посадка/перемещение по планетам, добыча ресурсов, строительство построек/кораблей, прогрессия и торговля.
- **Движок:** Godot 4.7.2 (для C# — .NET-сборка).
- **Целевая платформа MVP:** Windows (x86_64).

2. Масштабирование по этапам

Этап A — MVP (3–6 месяцев)

Цель: проверить основную петлю «летай → добывай → строй → улучшай».

- 1 управляемый корабль и 1 персонаж (от первого/третьего лица).
- 1 планета + орбита + небольшой космический сектор.
- Базовая добыча ресурсов (несколько типов).
- Простое строительство: размещение готовых модулей/блоков на сетке (без полноценной воксельной деформации).
- Инвентарь, крафт, сохранение/загрузка.
- Базовый UI, звук, туториал.

Этап B — Alpha (6–12 месяцев)

- Несколько планет/локаций, более быстрые перелёты между ними.
- Расширенное строительство (иерархия конструкций, энергия/топливо, функциональные блоки).
- Простая экономика (NPC-торговцы, цены, контракты).
- Опасности/бой (PVE), повреждения и ремонт.
- Оптимизация стриминга мира и производительности.

Этап C — Multiplayer/расширение (долгосрочно)

- Кооп/выделенные сессии (Godot Multiplayer API / ENet / WebSocket).
- Общие постройки, торговля между игроками.
- Более сложная физика конструкций, большие станции/корабли.
- Персистентность мира на сервере, античит, баланс.

3. Технический план на Godot 4.7.2

3.1 Архитектура проекта

- **Структура:**
  - `scenes/player`, `scenes/ships`, `scenes/world`, `scenes/buildables`, `scenes/ui`
  - `scripts/core` (состояние, события), `scripts/systems` (полёт, добыча, стройка)
  - `resources/` (предметы, рецепты, блоки как `Resource`)
  - `autoload/` — `GameState`, `EventBus`, `SaveSystem`, `WorldManager`
- **Язык:** C# рекомендуется для сложных систем и будущего серверного кода; GDScript допустим для прототипа/UI.
- **Данные:** использовать `Resource` / `.tres` для предметов, рецептов, конфигов блоков.

3.2 Ключевые системы

- **Полёт и управление:**
  - Режимы: космос (6DoF/инерция) и поверхность (персонаж + гравитация).
  - Переключение «корабль ↔ персонаж», посадка/взлёт.

- **Мир и масштаб:**
  - Секторная/чанковая структура пространства и поверхностей.
  - Стриминг сцен чанков, LOD, пулинг объектов.
  - **Floating origin / перецентрирование** для больших дистанций (критично для космоса).
- **Строительство:**
  - MVP: сеточное размещение модулей с проверкой коллизий и снапом.
  - Позже: составные конструкции, сохранение «блюпринтов», расчёт массы/энергии.
  - Полноценные воксели — отдельный сложный этап; на старте избегать.
- **Ресурсы и экономика:**
  - Узлы добычи, инструменты, инвентарь, крафт по рецептам.
  - Баланс ввода/вывода ресурсов, хранилища.
- **Сохранения:**
  - Сериализация состояния игрока, инвентаря и изменений в чанках/постройках.

4. Дорожная карта (первые шаги)

1. **Неделя 1 — Подготовка:** создать проект *Space Universe* в Godot 4.7.2, настроить Git и структуру папок; определить GDScript/C#, настроить Input Map (полёт, взаимодействие, стройка).
2. **Недели 2–4 — Прототип полёта:** космическая сцена, управляемый корабль, камера, HUD скорости/направления; базовая планета-заглушка и переход к её орбите.
3. **Недели 5–8 — Добыча и инвентарь:** ресурсные объекты, инструмент добычи, инвентарь, простой крафт.
4. **Недели 9–12 — Строительство MVP:** размещение/удаление модулей на сетке, сохранение построек.
5. **Недели 13+ — Играбельный цикл и полировка:** цели/задания, туториал, оптимизация, сборка тестового билда.

5. Критические риски

- **Слишком большой скоуп:** не пытаться сразу сделать воксели, MMO и бесшовную вселенную.

- **Производительность открытого мира:** закладывать стриминг, LOD и пулинг с самого начала.
- **Сложность строительства и физики:** начинать с модульной сетки, усложнять итеративно.
- **Сетевой код:** отложить полноценный мультиплеер до стабильного single-player ядра.

Итоговый ориентир: сначала сделать маленький, но законченный и приятный в управлении космический MVP на Godot 4.7.2, а уже затем расширять мир по секторам.